

ST. JOSEPH'S UNIVERSITY

Lalbagh Road, Bengaluru-560027

SCHOOL OF INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

LABORATORY RECORD

OF

Deep Learning Lab

(CA 8P125)



MASTERS OF COMPUTER APPLICATIONS

(MCA)

Submitted by:

ANTHONY PINTO ROBINSON

(253MCA37)

ST. JOSEPH'S UNIVERSITY

Lalbagh Road, Bengaluru-560027



This is to certify that **ANTHONY PINTO ROBINSON (253MCA37)** has satisfactorily completed the laboratory assignments prescribed for the **Deep Learning Lab** at **St. Joseph's University**, as part of the **Third Semester Master of Computer Applications (MCA)** degree programme for the academic year **2026–27**.

FACULTY IN-CHARGE

HOD / PG CO-ORDINATOR

DATE:

Examiner's Sign:

1. _____

2. _____

INDEX

S No.	Title	Pg. No
1.	Activation Functions – Sigmoid, Tanh, ReLU and Softmax	1-3
2.	Single Layer Perceptron-AND & OR	4-5
3.	Gradient Descent for Linear Regression	6
4.	Two-Layer Neural Network using NumPy	7-8
5.	Two-Layer Neural Network - Backpropagation	9-10
6.	Radial Basis Function (RBF) Network	11-12
7.	Learning Vector Quantization (LVQ)	13-14
8.	Self-Organizing Map (SOM)	15-16
9.	Adaptive Resonance Theory (ART)	17-18
10.	Neural Network with Dropout	19-21
11.	Competitive Learning for Unsupervised Classification	22-23

12.	Convolutional Neural Network (CNN)	24-26
13.	VGG16/ResNET50 Image Classification	27-28
14.	Time Series Prediction	29
15.	GRU & LSTM Model –Performance Comparison	30-31

Exp No.: 01

Implement Sigmoid, Tanh, ReLU, Softmax and Plot graphs and compare outputs.

Aim:

To implement and compare Sigmoid, Tanh, ReLU, and Softmax activation functions using Python and plot their outputs.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def tanh(x):
    return np.tanh(x)
def relu(x):
    return np.maximum(0, x)
def softmax(x):
    exp_x = np.exp(x - np.max(x))
    return exp_x / np.sum(exp_x)

# Input values
x = np.array([-2, -1, 0, 1, 2])
while True:
    print("\n===== ACTIVATION FUNCTION MENU =====")
    print("1. Sigmoid")
    print("2. Tanh")
    print("3. ReLU")
    print("4. Softmax")
    print("5. Compare All Outputs")
    print("6. Plot All Graphs")
    print("7. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        print("Sigmoid Output:")
        print(sigmoid(x))
    elif choice == 2:
        print("Tanh Output:")
        print(tanh(x))
    elif choice == 3:
        print("ReLU Output:")
        print(relu(x))
    elif choice == 4:
        print("Softmax Output:")
        print(softmax(x))
    elif choice == 5:
        print("\nInput:", x)
        print("Sigmoid :", sigmoid(x))
```

```

        print("Tanh :", tanh(x))
        print("ReLU :", relu(x))
        print("Softmax :", softmax(x))
    elif choice == 6:
        x_plot = np.linspace(-5, 5, 100)
        plt.plot(x_plot, sigmoid(x_plot), label="Sigmoid")
        plt.plot(x_plot, tanh(x_plot), label="Tanh")
        plt.plot(x_plot, relu(x_plot), label="ReLU")
        plt.plot(x_plot, softmax(x_plot), label="Softmax")
        plt.xlabel("Input")
        plt.ylabel("Output")
        plt.title("Comparison of Activation Functions")
        plt.legend()
        plt.grid()
        plt.show()
    elif choice == 7:
        print("Program ended.")
        break
    else:
        print("Invalid choice! Please try again.")

```

Output:

```

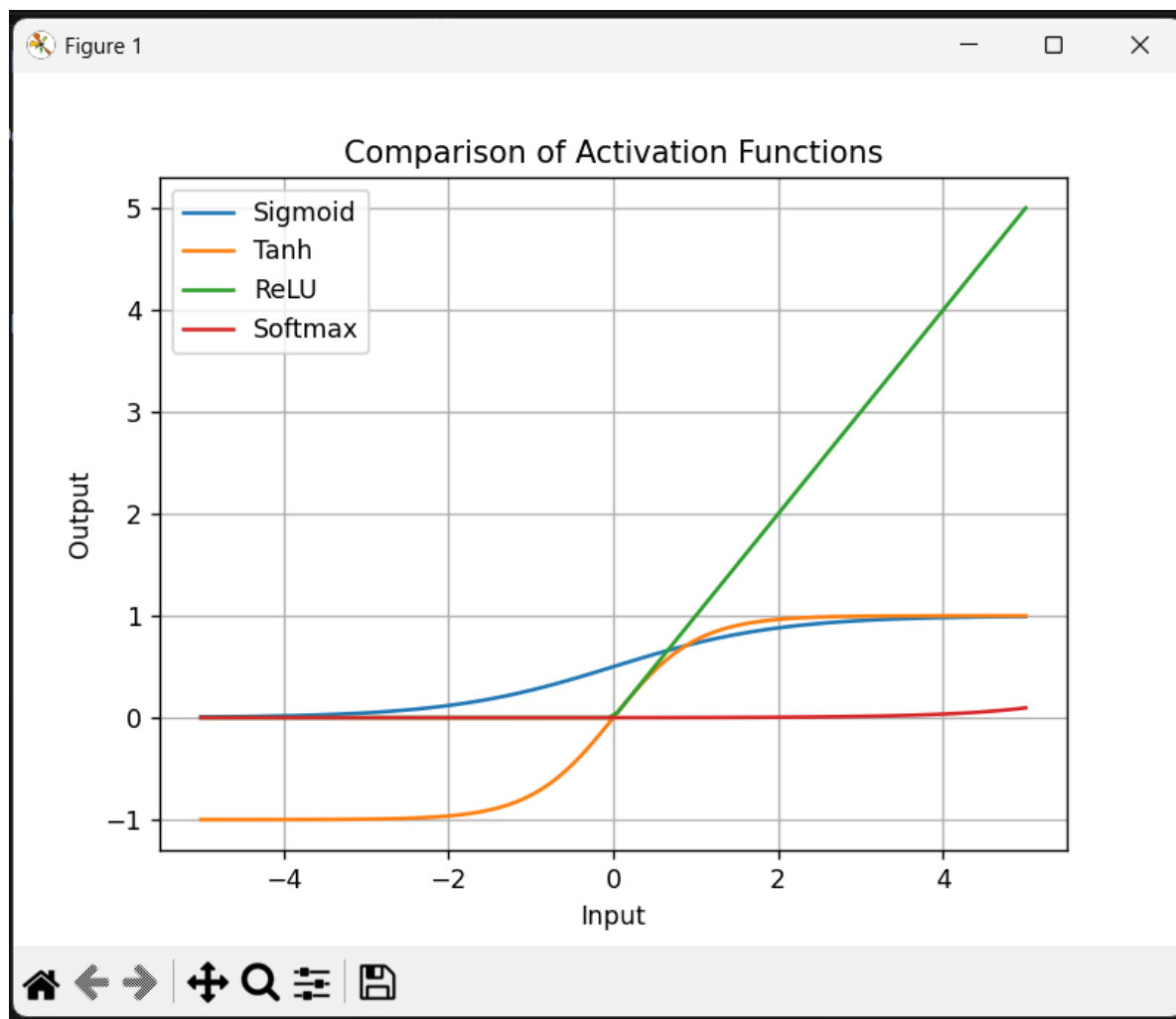
PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg1.py

===== ACTIVATION FUNCTION MENU =====
1. Sigmoid
2. Tanh
3. ReLU
4. Softmax
5. Compare All Outputs
6. Plot All Graphs
7. Exit
Enter your choice: 5

Input: [-2 -1  0  1  2]
Sigmoid : [0.11920292 0.26894142 0.5          0.73105858 0.88079708]
Tanh     : [-0.96402758 -0.76159416 0.          0.76159416 0.96402758]
ReLU     : [0 0 0 1 2]
Softmax  : [0.01165623 0.03168492 0.08612854 0.23412166 0.63640865]

===== ACTIVATION FUNCTION MENU =====
1. Sigmoid
2. Tanh
3. ReLU
4. Softmax
5. Compare All Outputs
6. Plot All Graphs
7. Exit
Enter your choice: 6

```



Exp No.: 02

Implement Single Layer Perceptron for AND/OR classification.

Aim:

To implement a single-layer perceptron to classify logic gates (AND/OR) using a simple weight update rule.

Code:

```
import numpy as np

def perceptron_train(X, y, epochs=10, lr=0.1):
    weights = np.zeros(X.shape[1])
    bias = 0
    for _ in range(epochs):
        for i in range(X.shape[0]):
            y_pred = 1 if (np.dot(X[i], weights) + bias) > 0 else 0
            weights += lr * (y[i] - y_pred) * X[i]
            bias += lr * (y[i] - y_pred)
    return weights, bias

def main():
    X = np.array([[0,0], [0,1], [1,0], [1,1]])
    y_and = np.array([0, 0, 0, 1])
    y_or = np.array([0, 1, 1, 1])

    while True:
        print("===== SINGLE LAYER PERCEPTRON =====")
        print("1. AND Gate")
        print("2. OR Gate")
        print("3. Exit")
        choice = input("Enter your choice: ")

        if choice == '1':
            w_and, b_and = perceptron_train(X, y_and)
            print("\n AND Gate Results")
            print("-----")
            print("Input  Output")
            for i in range(len(X)):
                print(f'{X[i]}  {y_and[i]}')
            print(f'\nWeights: {w_and}')
            print(f'Bias: {b_and}\n')
        elif choice == '2':
            w_or, b_or = perceptron_train(X, y_or)
            print("\n OR Gate Results")
            print("-----")
            print("Input  Output")
            for i in range(len(X)):
                print(f'{X[i]}  {y_or[i]}')
            print(f'\nWeights: {w_or}')
```

```

        print(f'Bias: {b_or}\n')
    elif choice == '3':
        break

if __name__ == "__main__":
    main()

```

Output:

```

PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg2.py
===== SINGLE LAYER PERCEPTRON =====
1. AND Gate
2. OR Gate
3. Exit
Enter your choice: 1

AND Gate Results
-----
Input Output
[0 0]    0
[0 1]    0
[1 0]    0
[1 1]    1

Weights: [0.2 0.1]
Bias: -0.2

===== SINGLE LAYER PERCEPTRON =====
1. AND Gate
2. OR Gate
3. Exit
Enter your choice: 2

OR Gate Results
-----
Input Output
[0 0]    0
[0 1]    1
[1 0]    1
[1 1]    1

Weights: [0.1 0.1]
Bias: 0.0

```

Exp No.: 03

Implement Gradient Descent for Linear Regression (from scratch).

Aim:

To apply gradient descent to compute parameters for linear regression models from scratch.

Code:

```
# Gradient Descent for Linear Regression
# Data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
# Initial values
m = 0    # slope
b = 0    # intercept
lr = 0.01 # learning rate
epochs = 1000
n = len(x)
# Gradient Descent
for i in range(epochs):
    # Predictions
    y_pred = [m * xi + b for xi in x]
    # Calculate gradients
    dm = (-2/n) * sum(x[j] * (y[j] - y_pred[j]) for j in range(n))
    db = (-2/n) * sum(y[j] - y_pred[j] for j in range(n))
    # Update m and b
    m = m - lr * dm
    b = b - lr * db
# Final results
print("Slope (m):", m)
print("Intercept (b):", b)

print("\nPredictions:")
for xi in x:
    print("x =", xi, "y =", m * xi + b)
```

Output:

```
PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg3.py
Slope (m): 1.9951803506719779
Intercept (b): 0.017400463340610635

Predictions:
x = 1 y = 2.0125808140125887
x = 2 y = 4.007761164684567
x = 3 y = 6.002941515356544
x = 4 y = 7.998121866028522
x = 5 y = 9.9933022167005
```

Exp No.: 04

Implement 2-layer Neural Network using NumPy.

Aim:

To construct a basic Adaline 2-layer neural network from scratch utilizing numpy functions for weights updating, forward passing, backward passing and MSE loss computation.

Code:

```
import numpy as np

# Loan data
# [Income, Credit Score, Loan Amount]
X = np.array([
    [50, 700, 20],
    [60, 750, 25],
    [30, 600, 30],
    [40, 650, 35],
    [80, 800, 20],
    [25, 550, 40]
], dtype=float)

# 1 = Approved, 0 = Rejected
Y = np.array([[1], [1], [0], [0], [1], [0]])

# Normalize data
X = X / np.max(X, axis=0)

# Initialize weights
np.random.seed(1)
W1 = np.random.rand(3, 4)
b1 = np.zeros((1, 4))
W2 = np.random.rand(4, 1)
b2 = np.zeros((1, 1))

lr = 0.1

# Training
for epoch in range(1000):
    # ----- Forward Pass -----
    Z1 = np.dot(X, W1) + b1
```

```

H = 1 / (1 + np.exp(-Z1))
Z2 = np.dot(H, W2) + b2
O = 1 / (1 + np.exp(-Z2))
# ----- Loss -----
loss = np.mean((Y - O) ** 2)
# ----- Backward Pass -----
dO = (O - Y) * O * (1 - O)
dW2 = np.dot(H.T, dO)
db2 = np.sum(dO, axis=0, keepdims=True)
dH = np.dot(dO, W2.T)
dZ1 = dH * H * (1 - H)
dW1 = np.dot(X.T, dZ1)
db1 = np.sum(dZ1, axis=0, keepdims=True)
# ----- Weight Update -----
W2 = W2 - lr * dW2
b2 = b2 - lr * db2
W1 = W1 - lr * dW1
b1 = b1 - lr * db1
# Prediction
prediction = (O >= 0.5).astype(int)
print("Predicted Classes:")
print(prediction.flatten())
print("\nActual Classes:")
print(Y.flatten())
print("\nFinal Loss:", loss)

```

Output:

```

PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg4.py
Predicted Classes:
[1 1 0 0 1 0]

Actual Classes:
[1 1 0 0 1 0]

Final Loss: 0.023389992250773105

```

Exp No.: 05

Two layer neural network using forward pass loss back propagation weight update

Aim:

To construct a Multilayer Perceptron from scratch to solve XOR using backward propagation rule to adjust weights.

Code:

```
import numpy as np

# Input data (XOR)
X = np.array([[0,0], [0,1], [1,0], [1,1]])
Y = np.array([[0], [1], [1], [0]])

# Weights and biases
W1 = np.random.rand(2, 2)
b1 = np.zeros((1, 2))

W2 = np.random.rand(2, 1)
b2 = np.zeros((1, 1))

lr = 0.1

# Training
for i in range(1000):

    # 1. Forward Pass
    H = np.tanh(np.dot(X, W1) + b1)
    O = 1 / (1 + np.exp(-(np.dot(H, W2) + b2)))

    # 2. Loss
    loss = np.mean((Y - O) ** 2)
```

```

# 3. Backpropagation
dO = (O - Y) * O * (1 - O)

dW2 = np.dot(H.T, dO)
db2 = np.sum(dO, axis=0, keepdims=True)

dH = np.dot(dO, W2.T)
dH = dH * (1 - H ** 2)

dW1 = np.dot(X.T, dH)
db1 = np.sum(dH, axis=0, keepdims=True)

# 4. Weight Update
W2 -= lr * dW2
b2 -= lr * db2
W1 -= lr * dW1
b1 -= lr * db1

print("Final Loss:", loss)
print("Predicted Output:")
print(np.round(O, 2))

```

Output:

```

PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg5.py
Final Loss: 0.05619836282822847
Predicted Output:
[[0.21]
 [0.79]
 [0.79]
 [0.31]]

Predicted Classes:
[[0]
 [1]
 [1]
 [0]]

Actual Classes:
[[0]
 [1]
 [1]
 [0]]

```

Exp No.: 06

Implement Radial Basis Function (RBF) Network

Aim:

To construct a Radial Basis Function Neural Network using Numpy.

Code:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score

# Load Iris dataset

iris = load_iris()

X = iris.data

y = iris.target

# Scale data

scaler = StandardScaler()

X = scaler.fit_transform(X)

# Split data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Choose centers

centers = X_train[:3]

sigma = 1.0

# RBF function

def rbf(x, center):

    return np.exp(-np.sum((x - center) ** 2) / (2 * sigma ** 2))

# Create RBF features

def rbf_features(X):

    R = np.zeros((len(X), len(centers)))

    for i in range(len(X)):
```

```

        for j in range(len(centers)):
            R[i, j] = rbf(X[i], centers[j])

    return R

# Transform data
R_train = rbf_features(X_train)
R_test = rbf_features(X_test)

# Train output weights
W = np.linalg.pinv(R_train).dot(y_train)

# Predict
pred = np.round(R_test.dot(W)).astype(int)

# Accuracy
accuracy = accuracy_score(y_test, pred)

print("Predicted Classes:")
print(pred)
print("\nActual Classes:")
print(y_test)

print("\nAccuracy:", accuracy)

```

Output:

```

PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg6.py
Predicted Classes:
[1 0 0 2 2 0 1 1 0 1 2 0 0 0 0 2 1 0 1 1 0 2 0 1 0 1 1 1 0 0]

Actual Classes:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

Accuracy: 0.5333333333333333

```

Exp No.: 07

Implement Learning Vector Quantization (LVQ)

Aim:

To implement Learning Vector Quantization algorithm for pattern classification by adjusting nearest class prototypes.

Code:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

# Load Iris dataset

iris = load_iris()

X = iris.data

y = iris.target

# Scale data

X = StandardScaler().fit_transform(X)

# Split data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Initial prototypes: one for each class

W = np.array([
    X_train[y_train == 0][0],
    X_train[y_train == 1][0],
    X_train[y_train == 2][0]
])

labels = np.array([0, 1, 2])

lr = 0.1

# Training

for epoch in range(20):
    for i in range(len(X_train)):
```

```

# Calculate distance
distance = np.linalg.norm(W - X_train[i], axis=1)

# Find winner
winner = np.argmin(distance)

# Update prototype
if labels[winner] == y_train[i]:
    W[winner] += lr * (X_train[i] - W[winner])
else:
    W[winner] -= lr * (X_train[i] - W[winner])

# Prediction
pred = []
for x in X_test:
    distance = np.linalg.norm(W - x, axis=1)
    winner = np.argmin(distance)
    pred.append(labels[winner])

# Accuracy
accuracy = accuracy_score(y_test, pred)
print("Predicted Classes:")
print(pred)
print("\nActual Classes:")
print(y_test)
print("\nAccuracy:", accuracy)

```

Output:

```

PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg7.py
Predicted Classes:
[np.int64(1), np.int64(0), np.int64(2), np.int64(1), np.int64(1), np.int64(0), np.int64(1), np.int64(2), np.
int64(1), np.int64(1), np.int64(2), np.int64(0), np.int64(0), np.int64(0), np.int64(0), np.int64(2), np.int6
4(2), np.int64(1), np.int64(1), np.int64(2), np.int64(0), np.int64(2), np.int64(0), np.int64(2), np.int64(2)
, np.int64(2), np.int64(2), np.int64(2), np.int64(0), np.int64(0)]

Actual Classes:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

Accuracy: 0.9666666666666667

```

Exp No.: 08

Implement Self Organizing Map (SOM)

Aim:

To implement a simple Self-Organizing Map (SOM) to learn topologies from random dataset values.

Code:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

# Load Iris dataset

iris = load_iris()

X = iris.data

# Normalize data

X = StandardScaler().fit_transform(X)

# SOM size

rows = 2

cols = 2

# Initialize weights

weights = np.random.rand(rows, cols, 4)

learning_rate = 0.5

# Training

for epoch in range(100):

    for x in X:

        # Find Best Matching Unit (BMU)

        distance = np.sum((weights - x) ** 2, axis=2)

        bmu = np.unravel_index(np.argmin(distance), distance.shape)

        # Update BMU

        r, c = bmu

        weights[r, c] += learning_rate * (x - weights[r, c])

# Display SOM mapping

print("SOM Cluster Mapping:")

for i, x in enumerate(X):
```

```
distance = np.sum((weights - x) ** 2, axis=2)

bmu = np.unravel_index(np.argmin(distance), distance.shape)

print("Sample", i + 1, "-> Node", bmu)
```

Output:

```
PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg8.py
SOM Cluster Mapping:
Sample 1 -> Node (np.int64(1), np.int64(1))
Sample 2 -> Node (np.int64(1), np.int64(1))
Sample 3 -> Node (np.int64(1), np.int64(1))
Sample 4 -> Node (np.int64(1), np.int64(1))
Sample 5 -> Node (np.int64(1), np.int64(1))
Sample 6 -> Node (np.int64(1), np.int64(1))
Sample 7 -> Node (np.int64(1), np.int64(1))
Sample 8 -> Node (np.int64(1), np.int64(1))
Sample 9 -> Node (np.int64(1), np.int64(1))
Sample 10 -> Node (np.int64(1), np.int64(1))
Sample 11 -> Node (np.int64(1), np.int64(1))
Sample 12 -> Node (np.int64(1), np.int64(1))
Sample 13 -> Node (np.int64(1), np.int64(1))
Sample 14 -> Node (np.int64(1), np.int64(1))
Sample 15 -> Node (np.int64(1), np.int64(1))
Sample 16 -> Node (np.int64(1), np.int64(1))
Sample 17 -> Node (np.int64(1), np.int64(1))
Sample 18 -> Node (np.int64(1), np.int64(1))
Sample 19 -> Node (np.int64(1), np.int64(1))
Sample 20 -> Node (np.int64(1), np.int64(1))
Sample 21 -> Node (np.int64(1), np.int64(1))
Sample 22 -> Node (np.int64(1), np.int64(1))
Sample 23 -> Node (np.int64(1), np.int64(1))
Sample 24 -> Node (np.int64(1), np.int64(1))
Sample 25 -> Node (np.int64(1), np.int64(1))
Sample 26 -> Node (np.int64(1), np.int64(1))
Sample 27 -> Node (np.int64(1), np.int64(1))
Sample 28 -> Node (np.int64(1), np.int64(1))
Sample 29 -> Node (np.int64(1), np.int64(1))
Sample 30 -> Node (np.int64(1), np.int64(1))
Sample 31 -> Node (np.int64(1), np.int64(1))
Sample 32 -> Node (np.int64(1), np.int64(1))
Sample 33 -> Node (np.int64(1), np.int64(1))
Sample 34 -> Node (np.int64(1), np.int64(1))
Sample 35 -> Node (np.int64(1), np.int64(1))
Sample 36 -> Node (np.int64(1), np.int64(1))
Sample 37 -> Node (np.int64(1), np.int64(1))
Sample 38 -> Node (np.int64(1), np.int64(1))
Sample 39 -> Node (np.int64(1), np.int64(1))
Sample 40 -> Node (np.int64(1), np.int64(1))
Sample 41 -> Node (np.int64(1), np.int64(1))
Sample 42 -> Node (np.int64(1), np.int64(1))
Sample 43 -> Node (np.int64(1), np.int64(1))
Sample 44 -> Node (np.int64(1), np.int64(1))
Sample 45 -> Node (np.int64(1), np.int64(1))
Sample 46 -> Node (np.int64(1), np.int64(1))
Sample 47 -> Node (np.int64(1), np.int64(1))
Sample 48 -> Node (np.int64(1), np.int64(1))
Sample 49 -> Node (np.int64(1), np.int64(1))
Sample 50 -> Node (np.int64(1), np.int64(1))
```

Exp No.: 09

Implement Adaptive Resonance Theory (ART)

Aim:

To implement ART-1 logic algorithm capable of continuous learning of binary patterns.

Code:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.preprocessing import MinMaxScaler

# Load Iris dataset

iris = load_iris()

X = iris.data

# Scale values between 0 and 1

X = MinMaxScaler().fit_transform(X)

# Parameters

vigilance = 0.7

learning_rate = 0.5

# Start with no clusters

weights = []

# ART training

for x in X:

    # Create first cluster

    if len(weights) == 0:

        weights.append(x.copy())

        continue

    # Calculate similarity

    similarity = []

    for w in weights:

        s = np.sum(np.minimum(x, w)) / np.sum(x)

        similarity.append(s)

    # Find best matching cluster

    winner = np.argmax(similarity)

    # Vigilance test
```

```

if similarity[winner] >= vigilance:
    # Update cluster
    weights[winner] = (
        learning_rate * x +
        (1 - learning_rate) * weights[winner]
    )
else:
    # Create new cluster
    weights.append(x.copy())

# Display result
print("Number of Clusters:", len(weights))
print("\nFinal Cluster Weights:")
for i, w in enumerate(weights):
    print("Cluster", i + 1, ":", np.round(w, 2))

```

Output:

```

PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg9.py
Number of Clusters: 7

Final Cluster Weights:
Cluster 1 : [0.05 0.45 0.04 0.02]
Cluster 2 : [0.12 0.36 0.06 0.07]
Cluster 3 : [0.08 0.51 0.05 0.04]
Cluster 4 : [0.2  0.59 0.08 0.05]
Cluster 5 : [0.24 0.14 0.43 0.41]
Cluster 6 : [0.36 0.31 0.49 0.48]
Cluster 7 : [0.51 0.45 0.71 0.79]

```

Exp No.: 10

Train neural network with and without Dropout, Compare accuracy and loss curves.

Aim:

To observe the effect of adding dropout layers in Keras Neural Networks on validation accuracy.

Code:

```
import tensorflow as tf
import matplotlib.pyplot as plt

# Load MNIST dataset
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize data
X_train = X_train / 255.0
X_test = X_test / 255.0

# ----- Without Dropout -----
model1 = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model1.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

history1 = model1.fit(X_train, y_train,
                     epochs=5,
                     validation_data=(X_test, y_test))

# ----- With Dropout -----
model2 = tf.keras.Sequential([
```

```

tf.keras.layers.Flatten(input_shape=(28, 28)),
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dropout(0.5),
tf.keras.layers.Dense(10, activation='softmax')
])
model2.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

history2 = model2.fit(X_train, y_train,
                     epochs=5,
                     validation_data=(X_test, y_test))

# ----- Accuracy Comparison -----
plt.plot(history1.history['val_accuracy'], label='Without Dropout')
plt.plot(history2.history['val_accuracy'], label='With Dropout')

plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Accuracy Comparison')
plt.legend()
plt.show()

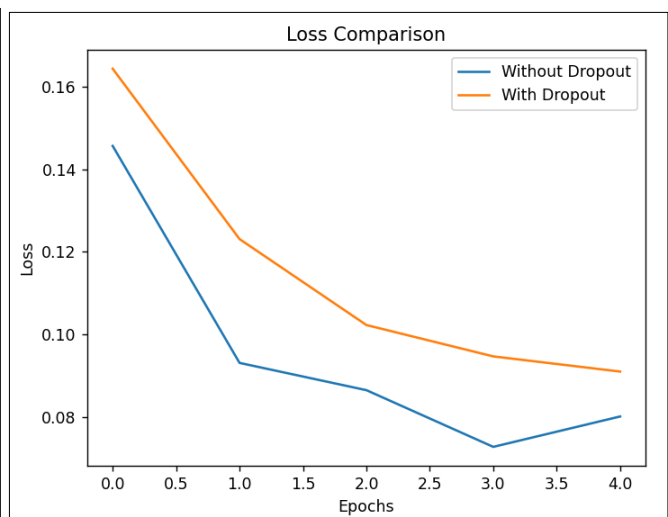
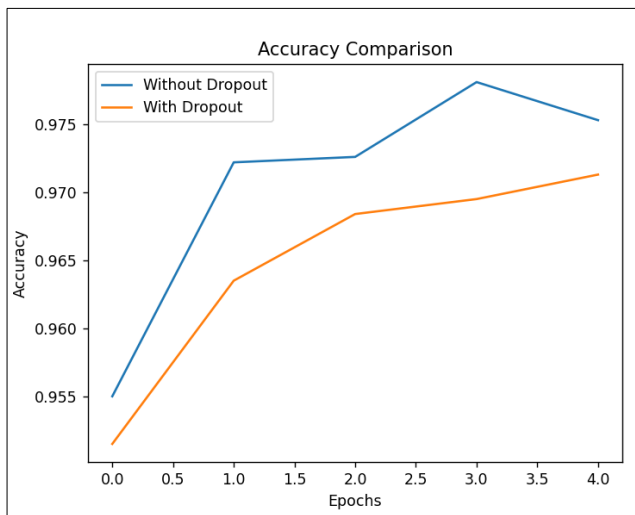
# ----- Loss Comparison -----
plt.plot(history1.history['val_loss'], label='Without Dropout')
plt.plot(history2.history['val_loss'], label='With Dropout')

plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Loss Comparison')
plt.legend()
plt.show()

```

Output:

```
Epoch 1/5
1875/1875 — 7s 3ms/step - accuracy: 0.9276 - loss: 0.2555 - val_accuracy: 0.9550 - val_loss: 0.1457
Epoch 2/5
1875/1875 — 7s 4ms/step - accuracy: 0.9675 - loss: 0.1097 - val_accuracy: 0.9722 - val_loss: 0.0931
Epoch 3/5
1875/1875 — 16s 8ms/step - accuracy: 0.9777 - loss: 0.0747 - val_accuracy: 0.9726 - val_loss: 0.0865
Epoch 4/5
1875/1875 — 13s 7ms/step - accuracy: 0.9833 - loss: 0.0545 - val_accuracy: 0.9781 - val_loss: 0.0728
Epoch 5/5
1875/1875 — 16s 4ms/step - accuracy: 0.9873 - loss: 0.0417 - val_accuracy: 0.9753 - val_loss: 0.0801
Epoch 1/5
1875/1875 — 8s 4ms/step - accuracy: 0.8827 - loss: 0.3956 - val_accuracy: 0.9515 - val_loss: 0.1644
Epoch 2/5
1875/1875 — 6s 3ms/step - accuracy: 0.9338 - loss: 0.2216 - val_accuracy: 0.9635 - val_loss: 0.1231
Epoch 3/5
1875/1875 — 7s 4ms/step - accuracy: 0.9460 - loss: 0.1839 - val_accuracy: 0.9684 - val_loss: 0.1023
Epoch 4/5
1875/1875 — 5s 3ms/step - accuracy: 0.9507 - loss: 0.1642 - val_accuracy: 0.9695 - val_loss: 0.0947
Epoch 5/5
1875/1875 — 7s 4ms/step - accuracy: 0.9559 - loss: 0.1466 - val_accuracy: 0.9713 - val_loss: 0.0910
```



Exp No.: 11

Implement competitive learning for unsupervised classification.

Aim:

To construct a competitive learning scheme which clusters unstructured numerical inputs efficiently via distance rules.

Code:

```
import numpy as np

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load Iris dataset
iris = load_iris()
X = iris.data

# Normalize data
X = StandardScaler().fit_transform(X)

# Initialize 3 cluster centers
weights = X[:3].copy()
learning_rate = 0.1

# Training
for epoch in range(20):
    for x in X:
        # Calculate distance
        distance = np.linalg.norm(weights - x, axis=1)

        # Find winning neuron
        winner = np.argmin(distance)

        # Update winner
        weights[winner] = weights[winner] + learning_rate * (
            x - weights[winner]
        )

# Display clusters
print("Final Cluster Centers:")
print(np.round(weights, 2))
print("\nCluster Assignment:")
```

```
for i, x in enumerate(X[:10]):  
    distance = np.linalg.norm(weights - x, axis=1)  
    winner = np.argmin(distance)  
    print("Sample", i + 1, "-> Cluster", winner + 1)
```

Output:

```
PS C:\Users\LENOVO\Documents\Projects\College Projects\Deep_Learning_Lab> python pg11.py  
Final Cluster Centers:  
[[ 1.1  0.17  1.04  1.28]  
 [ 0.09 -0.76  0.59  0.6 ]  
 [-1.07  0.77 -1.3  -1.25]]  
  
Cluster Assignment:  
Sample 1 -> Cluster 3  
Sample 2 -> Cluster 3  
Sample 3 -> Cluster 3  
Sample 4 -> Cluster 3  
Sample 5 -> Cluster 3  
Sample 6 -> Cluster 3  
Sample 7 -> Cluster 3  
Sample 8 -> Cluster 3  
Sample 9 -> Cluster 3  
Sample 10 -> Cluster 3
```

Exp No.: 12

Build CNN using TensorFlow/Keras, Evaluate accuracy and confusion matrix.

Aim:

To construct a Convolutional Neural Network on image sets and evaluate with a confusion matrix using tf/keras.

Code:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


# Load data
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()


# Normalize data
X_train = X_train / 255.0
X_test = X_test / 255.0


# Add channel dimension
X_train = X_train[..., np.newaxis]
X_test = X_test[..., np.newaxis]


# CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, 3, activation='relu',
                           input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

```
# Compile model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X_train, y_train, epochs=3)

# Test accuracy
loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy:", accuracy)

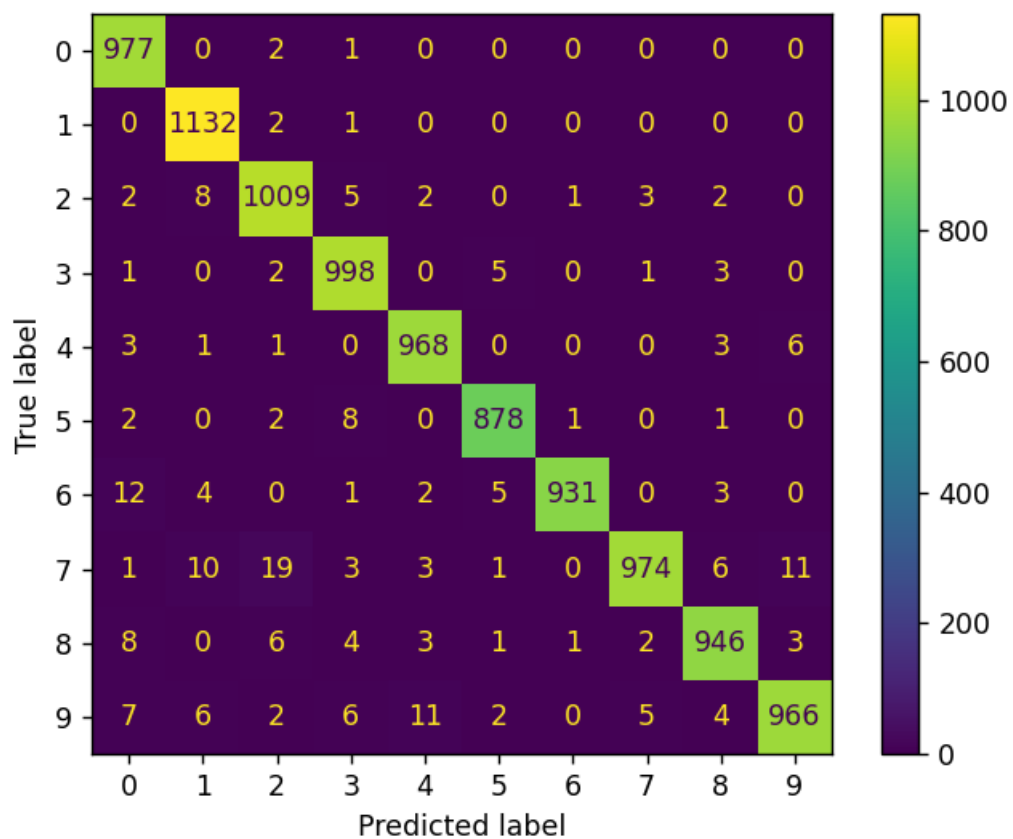
# Prediction
prediction = model.predict(X_test)
prediction = np.argmax(prediction, axis=1)

# Confusion matrix
cm = confusion_matrix(y_test, prediction)

print("Confusion Matrix:")
print(cm)

# Display
ConfusionMatrixDisplay(cm).plot()
plt.show()

Output:
```



```

Epoch 1/3
1875/1875 ————— 11s 6ms/step - accuracy: 0.9384 - loss: 0.2171
Epoch 2/3
1875/1875 ————— 11s 6ms/step - accuracy: 0.9771 - loss: 0.0787
Epoch 3/3
1875/1875 ————— 11s 6ms/step - accuracy: 0.9822 - loss: 0.0605
313/313 ————— 1s 3ms/step - accuracy: 0.9779 - loss: 0.0682
Accuracy: 0.9779000282287598
313/313 ————— 1s 2ms/step
Confusion Matrix:
[[ 977   0    2    1    0    0    0    0    0    0]
 [   0 1132    2    1    0    0    0    0    0    0]
 [   2    8 1009    5    2    0    1    3    2    0]
 [   1    0    2  998    0    5    0    1    3    0]
 [   3    1    1    0  968    0    0    0    3    6]
 [   2    0    2    8    0  878    1    0    1    0]
 [  12    4    0    1    2    5  931    0    3    0]
 [   1   10   19    3    3    1    0  974    6   11]
 [   8    0    6    4    3    1    1    2  946    3]
 [   7    6    2    6   11    2    0    5    4  966]]

```

Exp No.: 13

Apply VGG16 / ResNet50 for image classification (custom dataset).

Aim:

To employ pre-trained state of the art models such as VGG16 to extract features on custom images for classifications.

Code:

```
import numpy as np

import tensorflow as tf

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from tensorflow.keras.applications import VGG16

from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

from tensorflow.keras.models import Sequential

# Load Iris dataset

iris = load_iris()

X = iris.data

y = iris.target

# Normalize

X = X / X.max()

# Convert 4 features into a 2x2 image

X = X.reshape(150, 2, 2, 1)

# Convert grayscale to RGB

X = np.repeat(X, 3, axis=3)

# Resize images to 32x32

X = tf.image.resize(X, (32, 32))

# Convert TensorFlow tensor to NumPy

X = X.numpy()

# Split data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# VGG16
```

```

base_model = VGG16(
    weights=None,
    include_top=False,
    input_shape=(32, 32, 3)
)
# Build model
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(3, activation='softmax')
])
# Compile
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Train
model.fit(X_train, y_train, epochs=5, verbose=1)
# Test
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)

```

Output:

```

Epoch 1/5
4/4 ————— 5s 309ms/step - accuracy: 0.3417 - loss: 1.0988
Epoch 2/5
4/4 ————— 1s 323ms/step - accuracy: 0.3667 - loss: 1.3704
Epoch 3/5
4/4 ————— 1s 309ms/step - accuracy: 0.3333 - loss: 1.0995
Epoch 4/5
4/4 ————— 1s 312ms/step - accuracy: 0.3333 - loss: 1.0989
Epoch 5/5
4/4 ————— 1s 310ms/step - accuracy: 0.3333 - loss: 1.0989
1/1 ————— 0s 402ms/step - accuracy: 0.3333 - loss: 1.0985
Test Accuracy: 0.3333333432674408

```

Exp No.: 14

Predict next value in time-series (stock/sample dataset).

Aim:

To predict upcoming continuous values sequentially via Recurrent Neural Networks acting upon simple sine wave patterns.

Code:

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

# Sample time-series data

data = np.array([10, 12, 14, 16, 18, 20, 22, 24, 26, 28])

# Create input and output

X = data[:-1]

y = data[1:]

# Reshape input

X = X.reshape(-1, 1)

# Create model

model = Sequential([

    Dense(10, activation='relu', input_shape=(1,)),

    Dense(1)

])

# Compile

model.compile(optimizer='adam', loss='mse')

# Train

model.fit(X, y, epochs=100, verbose=0)

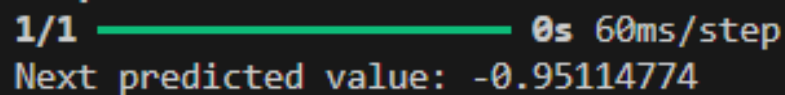
# Predict next value

last_value = np.array([[28]])

prediction = model.predict(last_value)

print("Next predicted value:", prediction[0][0])
```

Output:



```
1/1 ————— 0s 60ms/step
Next predicted value: -0.95114774
```

Exp No.: 15

Implement both GRU and LSTM models and compare performance.

Aim:

To compute and compare the final loss of Gated Recurrent Units versus Long Short Term Memory networks.

Code:

```
import numpy as np

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, GRU, Dense


# Sample data
data = np.array([10, 12, 14, 16, 18, 20, 22, 24, 26, 28,
                 30, 32, 34, 36, 38, 40])

# Create sequences
X = []
y = []

for i in range(len(data) - 3):
    X.append(data[i:i+3])
    y.append(data[i+3])

X = np.array(X)
y = np.array(y)

# Reshape for RNN
X = X.reshape((X.shape[0], X.shape[1], 1))

# ----- LSTM Model -----
lstm = Sequential([
    LSTM(10, input_shape=(3, 1)),
    Dense(1)
```

```

])

lstm.compile(optimizer='adam', loss='mse')

lstm_history = lstm.fit(X, y, epochs=50, verbose=0)

lstm_loss = lstm.evaluate(X, y, verbose=0)

# ----- GRU Model -----
gru = Sequential([
    GRU(10, input_shape=(3, 1)),
    Dense(1)
])

gru.compile(optimizer='adam', loss='mse')

gru_history = gru.fit(X, y, epochs=50, verbose=0)

gru_loss = gru.evaluate(X, y, verbose=0)

# ----- Compare -----
print("LSTM Loss:", lstm_loss)
print("GRU Loss:", gru_loss)

if lstm_loss < gru_loss:
    print("LSTM has lower loss.")
else:
    print("GRU has lower loss.")

```

Output:

```

LSTM Loss: 797.174560546875
GRU Loss: 743.6917114257812
GRU has lower loss.

```